COBTUPDT-cbl

Report date: *2026-10-29T17:39:56.701118596Z*

User ID: *374199896615:transform-demo*

AWS Account ID: *374199896615*

Table of Contents

1 High level overview....2

1.1 Overview....2

1.2 Key features....2

2 Input output....2

3 Data flow and dependencies....3

3.1 Data flow....3

3.2 Data dependencies....4

4 Error handling....5

AWS Transform uses generative AI. Use of AWS Transform is subject to AWS Responsible AI Policy https://aws.amazon.com/ai/responsible-ai/policy/

Documentation

1. High level overview

1.1. Overview

This program operates within a card transaction management system (CardDemo), serving as a batch maintenance utility for the Transaction Type reference table. It is part of the business logic layer responsible for keeping transaction classification data current and accurate. The program reads a sequential input file containing maintenance instructions and applies the corresponding database changes — inserts, updates, or deletes — to the CARDDEMO.TRANSACTION_TYPE table in DB2. It solves the problem of bulk-maintaining transaction type reference data without requiring manual, record-by-record intervention, enabling administrators to submit a batch file of changes to be processed automatically.

1.2. Key features

Feature	Description
Batch File Processing	Reads a sequential input file containing one or more transaction type maintenance records, processing each record in turn until end-of-file is reached.
Add Transaction Type	Inserts a new transaction type record into the CARDDEMO.TRANSACTION_TYPE table when the input record type is flagged as 'A'.
Update Transaction Type	Updates the description of an existing transaction type in the database when the input record type is flagged as 'U', matching on the transaction type number.
Delete Transaction Type	Removes a transaction type record from the database when the input record type is flagged as 'D', matching on the transaction type number.
Comment Line Support	Ignores input records prefixed with '*', allowing the input file to contain comment or documentation lines without causing errors.
SQL Error Handling	Evaluates DB2 SQLCODE after each database operation and triggers an abend routine with a descriptive error message if a negative SQLCODE or 'not found' condition is encountered.
Abend and Return Code Management	Sets a return code of 4 and displays an error message when an unrecoverable error occurs, signaling failure to the calling job control environment.
Input Validation	Validates the record type field on each input record and abends with an error message if an unrecognized operation type is encountered.

2. Input output

Name	Type	Description
INPFILE (TR-RECORD)	Read	Sequential input file containing transaction type records. Each record includes a record type indicator (A=Add, U=Update, D=Delete, *=Comment), a transaction type number (2 chars), and a transaction description (50 chars). The program reads all records sequentially to determine which database operations to perform.
CARDDEMO. TRANSACTION_TYPE	Read-Write	DB2 database table storing transaction type definitions with columns TR_TYPE (CHAR 2) and TR_DESCRIPTION (VARCHAR 50). The program inserts new records (type A), updates existing descriptions by TR_TYPE key (type U), and deletes records by TR_TYPE key (type D) based on the input file contents.

3. Data flow and dependencies

3.1. Data flow

Step	Process	Description	Records
1	Open Input File	Opens the sequential input file TR-RECORD (assigned to INPFILE) and checks file status for successful open	[WS-INF-STATUS, WS-INF-STAT1, WS-INF-STAT2]
2	Read First Record	Reads the first record from the sequential input file into WS-INPUT-REC before entering the main processing loop	[WS-INPUT-REC, INPUT-REC-TYPE, INPUT-REC-NUMBER, INPUT-REC-DESC, LASTREC]
3	Evaluate Record Type	Inspects INPUT-REC-TYPE field to determine the operation to perform: 'A' for insert, 'U' for update, 'D' for delete, '*' to skip commented lines, or abend on invalid type	[INPUT-REC-TYPE, WS-RETURN-MSG]
4	Insert Transaction Type Record	When INPUT-REC-TYPE is 'A', executes a DB2 INSERT into CARDDEMO.TRANSACTION_TYPE using TR_TYPE and TR_DESCRIPTION values from the input record. Abends on negative SQLCODE.	[INPUT-REC-NUMBER, INPUT-REC-DESC, SQLCODE, WS-VAR-SQLCODE, WS-RETURN-MSG]
5	Update Transaction Type Record	When INPUT-REC-TYPE is 'U', executes a DB2 UPDATE on CARDDEMO.TRANSACTION_TYPE setting TR_DESCRIPTION where TR_TYPE matches INPUT-REC-NUMBER. Abends on SQLCODE +100 (not found) or negative SQLCODE.	[INPUT-REC-NUMBER, INPUT-REC-DESC, SQLCODE, WS-VAR-SQLCODE, WS-RETURN-MSG]
6	Delete Transaction Type Record	When INPUT-REC-TYPE is 'D', executes a DB2 DELETE from CARDDEMO.TRANSACTION_TYPE where TR_TYPE matches INPUT-REC-NUMBER. Abends on SQLCODE +100 (not found) or negative SQLCODE.	[INPUT-REC-NUMBER, SQLCODE, WS-VAR-SQLCODE, WS-RETURN-MSG]
7	Read Next Record	Reads the next sequential record from TR-RECORD into WS-INPUT-REC. Sets LASTREC to	[WS-INPUT-REC, INPUT-REC-TYPE, INPUT-REC-NUMBER, INPUT-REC-DESC, LASTREC]

Step	Process	Description	Records
		'Y' when end-of-file is reached, terminating the processing loop.	
8	Close File and Stop	After all records are processed (LASTREC = 'Y'), closes the TR-RECORD input file and exits the program	[WS-INF-STATUS]
9	Abend Handling	On any error condition (invalid record type, DB2 error, or record not found), displays the error message stored in WS-RETURN-MSG and sets RETURN-CODE to 4 to signal abnormal termination to the calling environment	[WS-RETURN-MSG, RETURN-CODE]

3.2. Data dependencies

Dependency	Description
File Open Before Read	TR-RECORD input file must be successfully opened (WS-INF-STATUS = '00') before any READ operations are attempted. No explicit guard prevents reads if open fails, as the program continues regardless of open status.
LASTREC Flag State	The LASTREC flag controls the main processing loop. It must be initialized to spaces and only set to 'Y' upon AT END of the READ, ensuring all records are processed before loop termination.
INPUT-REC-TYPE Validity	All DB2 DML operations depend on INPUT-REC-TYPE being a valid value ('A', 'U', 'D', or '*'). An unrecognized type triggers an immediate abend via 9999-ABEND before any database operation is attempted.
INPUT-REC-NUMBER as DB2 Key	INPUT-REC-NUMBER (mapped to TR_TYPE column) is the primary key used in WHERE clauses for UPDATE and DELETE operations. Its accuracy is critical; an incorrect value will cause a +100 SQLCODE (not found) and abend.
DCLTRTYP Copybook DB2 Host Variable Declarations	The EXEC SQL INCLUDE DCLTRTYP copybook provides the DB2 host variable declarations for the TRANSACTION_TYPE table. The program's SQL statements depend on these declarations being correctly aligned with the actual DB2 table schema.
SQLCA Availability	The SQL Communication Area (SQLCA) must be included and populated by DB2 after each EXEC SQL statement. The program reads SQLCODE from SQLCA to determine success or failure of every database operation.
Sequential File Record Order Independence	Records in the input file are processed strictly in sequential order. There is no sorting or reordering, so the caller is responsible for ensuring that dependent operations (e.g., insert before update) appear in the correct sequence in the input file.
WS-INPUT-REC vs WS-INPUT-VARS Dual Structure	The FILE SECTION defines WS-INPUT-VARS as the FD record layout, but the READ statement uses INTO WS-INPUT-REC (a WORKING-STORAGE structure). Processing logic references INPUT-REC-TYPE/NUMBER/DESC from WS-INPUT-REC, making WS-INPUT-REC the authoritative source for all downstream operations.

4. Error handling

Category	Description
File Operation Errors	File open status is checked via WS-INF-STATUS after OPEN INPUT. A '00' status displays success; any other status displays 'OPEN FILE NOT OK' but does not halt execution or trigger an abend, meaning processing continues regardless of open failure. No explicit handling exists for READ errors beyond the AT END clause which sets LASTREC='Y' to terminate the read loop.
Input Record Type Validation Errors	The EVALUATE on INPUT-REC-TYPE handles valid types 'A', 'U', 'D', and '*' (comment lines). Any unrecognized type falls into the WHEN OTHER branch, which builds an error message 'ERROR: TYPE NOT VALID' into WS-RETURN-MSG and performs the 9999-ABEND paragraph to display the message and set RETURN-CODE to 4.
Database INSERT Errors	After the SQL INSERT, SQLCODE is evaluated. SQLCODE=0 indicates success. SQLCODE<0 (negative, indicating a DB2 error) builds an error message containing 'Error accessing: TRANSACTION_TYPE table. SQLCODE:' plus the formatted SQLCODE value into WS-RETURN-MSG, then performs 9999-ABEND. SQLCODE=+100 (not found) is not explicitly handled for INSERT.
Database UPDATE Errors	After the SQL UPDATE, SQLCODE is evaluated. SQLCODE=0 indicates success. SQLCODE=+100 (no rows found) builds 'No records found.' into WS-RETURN-MSG and performs 9999-ABEND. SQLCODE<0 (DB2 error) builds a detailed error message with the SQLCODE value into WS-RETURN-MSG and performs 9999-ABEND.
Database DELETE Errors	After the SQL DELETE, SQLCODE is evaluated. SQLCODE=0 indicates success. SQLCODE=+100 (no rows found) builds 'No records found.' into WS-RETURN-MSG and performs 9999-ABEND. SQLCODE<0 (DB2 error) builds a detailed error message with the SQLCODE value into WS-RETURN-MSG and performs 9999-ABEND.
Centralized Abend Handling (9999-ABEND)	All fatal errors route to the 9999-ABEND paragraph, which displays the contents of WS-RETURN-MSG and sets RETURN-CODE to 4 to signal failure to the calling environment. Notably, 9999-ABEND does not issue a STOP RUN, so execution continues after the abend paragraph returns, which is a significant deficiency — processing may proceed with subsequent records even after a fatal error.